

Codice su più Files

Compilazione, Test Driven Development e
Documentazione

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio

Proprietà di un SW

Proprietà di un Software

- Scrivere un programma C funzionante è sufficiente?
- Esempio:
 - obscure.c di [Stephen Dolan a IOCC](#) (International Obfuscated C Code Contest) 2024

```
long z,x,G;main(){for(puts("P1\n80
80"),scanf("%10lx",&G);3-z/2160;x=++z%81/8-
5)putchar(5>x?!(16>>(x^-(
(x<1))+1&G<<5>>z/648*5)^49:10);}
```

- Programma funzionante, ... ma:
 - Sapreste dire cosa fa?

(Alcune) Proprietà Desiderabili



- **Correttezza:** il programma soddisfa le specifiche
- **Affidabilità:** soddisfa le funzionalità intese
- **Usabilità:** lo sforzo necessario a impararlo, usarlo, preparare i dati e interpretare i risultati
- **Sicurezza:** non costituisce un rischio per l'ambiente o l'utilizzatore
- **Comprensibilità:** è facile da capire
- **Efficienza:** usa poche risorse (tempo, memoria, ...)
- **Manutenibilità:** è modificabile facilmente

La Correttezza è Sufficiente?

- **Corretto?** Sì (perché è stato valutato)
- **Usabile?** Poco
- **Sicuro?** Difficile a dirsi
- **Comprensibile?** No
- **Efficiente?** Non si può valutare
- **Mantenibile?** No

```
long z,x,G;main(){for(puts("P1\n80
80"),scanf("%10lx",&G);3-z/2160;x=++z%81/8-
5)putchar(5>x?!(16>>(x^-(
(x<1))+1&G<<5>>z/648*5)^49:10);}
```

- Per i curiosi provate a compilarlo ed eseguirlo:
 - `gcc obscure.c -o obscure -Wno-implicit-int -Wno-shift-op-parentheses -Wno-strict-prototypes -Wno-format -Wno-parentheses -Wno-unknown-warning-option -Wall -Wextra -pedantic -std=gnu17 -include stdio.h`
 - `echo faf6bdafe6 | ./obscure`

Coding Style



- Per una migliore leggibilità del codice è consigliabile seguire uno stile
 - Nessun stile standard per C
- Linux kernel coding style in breve:
 - Nessun «ritorno a capo» prima di { (tranne per la prima { di una funzione)
 - «Ritorno a capo» dopo } (tranne se continua la stessa espressione, e.g. «} else {»)
 - Niente righe di lunghezza infinita, cercate di stare in 80 caratteri
 - Guardate gli esempi del link
- Tool per formattazione automatica:
 - Lista: <https://github.com/camelomartins/awesome-linters>

Codice Autoesplicativo e Commentato



- Inoltre è consigliabile:
 1. Scegliere nomi di funzioni e variabili che siano (il più possibile) descrittivi e autoesplicativi.
 2. Scriveremo i commenti (che cosa contiene un file, che cosa fa una una funzione, ...) prima di scrivere il codice.
 3. Scriveremo il codice in modo che sia il più compatto e comprensibile possibile.
 4. Tornare sui punti 1..3 più volte durante lo sviluppo, migliorando nomi/commenti/codice fino a quando saremo ragionevolmente soddisfatti
- Formato (de-facto) standard per commenti:
 - [Doxygen](#)

Codice su più Files

Codice su più Files: Perché?

- In generale, è conveniente suddividere il codice sorgente in più file:
 - Migliora l'organizzazione del codice
 - Facilita il testing, la manutenzione, e la correzione degli errori
 - Permette di riutilizzare il codice per progetti diversi
- Un modo per suddividere il codice sorgente in più file:
 - Inclusione di header files e compilazione separata (visto in questo corso)
- Esistono anche altri due modi:
 - Creazione di librerie statiche o dinamiche
 - ~~Inclusione diretta di file sorgente e compilazione unica~~ — **DA NON FARE!**

Esempio 1

- Vogliamo scrivere un programma che:
 1. Legge un intero
 2. Calcola il quadrato
 3. Stampa il risultato
- Per facilitare riuso questi tre passi possono (e tipicamente sono) scritti come funzioni
 - 1 e 3 usano funzioni della libreria standard
 - `scanf()` per leggere in numero, `printf()` per stampare il risultato
 - 2 funzione nostra
 - Potrebbe essere utile per altri programmi (e.g., calcolo dell'ipotenusa di triangoli retti)
 - ... ma se scritta nello stesso file `.c` difficilmente riutilizzabile da altri programmi
 - Implementata in un **file sorgente** `.c` separato e dichiarata in un **file header** `.h`

Esempio 1 Cont.

- Scriviamo per prima cosa:
 - La dichiarazione della funzione che voglio scrivere `square.h`
- Poi:
 - Il codice dell'applicazione (dove c'è la funzione `main`) che la usa `mymain.c`
 - Il codice della funzione `square.c`



Due usi di `#include`. Tipicamente*:

- `"file.h"` cerca `file.h` nella cartella corrente
- `<file.h>` cerca `file.h` nella/e cartella/e di sistema

`square.h`

```
/** @brief Restituisce il quadrato del  
suo argomento  
*/  
int square(int n);
```

`mymain.c`

```
#include <stdio.h>  
#include "square.h"  
  
/** @brief Legge un intero e ne  
stampa il quadrato  
*/  
int main(void)  
{  
    int num;  
    int res;  
    scanf("%d", &num);  
    res = square(num);  
    printf("%d\n", res);  
}
```

#include

`square.c`

```
#include "square.h"  
  
int square(int n)  
{  
    return n * n;  
}
```

#include

*dipende dal compilatore (il comportamento non è specificato nello standard)

Come Generiamo il File Eseguibile?

1. Compiliamo separatamente i file sorgenti `.c` (generando un file `.o` per ogni file `.c`)

- `gcc -std=c18 -c square.c`
 - Produce: `square.o`
- `gcc -std=c18 -c mymain.c`
 - Produce: `mymain.o`

2. Linkiamo (collegiamo) i files `.o` (generando il file eseguibile `myprog`)

- `gcc -o myprog mymain.o square.o`

Nota:

- `-std=c18` specifica quale standard del linguaggio C usare:
 - Per brevità omissso nei lucidi rimanenti, ma è buona norma specificarlo

Vantaggio

- Se modifichiamo solo uno dei file sorgenti `.c`, e.g., `mymain.c` (resp. `square.c`), possiamo limitarci ai due comandi:
 1. `gcc -c mymain.c` (resp. `gcc -c square.c`)
 2. `gcc -o myprog mymain.o square.o`
- In generale `mymain.c` e `square.c` potrebbero essere molto grandi, quindi evitare di ricompilarli tutti risparmia tempo/energia/risorse... (cruciale fin dagli anni '70)
 - In generale avremo un file `mymain.c` e molteplici ($n \geq 0$) file `file1.c, ..., fileN.c`

Esempio 2

- Supponiamo che i requisiti dell'applicazione `myprog` siano cambiati:
 - Il numero intero di cui calcolare il quadrato non deve essere letto dallo standard input, ma fornito sulla riga di comando
 - Sarà quindi disponibile nel `main` come una stringa
- Dobbiamo sviluppare una funzione per convertire una stringa in un `int`
 - Chiameremo questa funzione `stoi()`
 - Che cosa significa `stoi`?

Esempio 2: Passo 1

- Assumiamo che un collega abbia scritto un file `stoi.h`, che specifica il comportamento della funzione `stoi()`
 - La specifica di una funzione:
 - Descrive **cosa** deve fare.
 - **Non come** fa a farlo.
 - Il collega ci fornisce:
 - `stoi.h` e `stoi.o`

`stoi.h`

```
/*
 * @brief Converte una stringa in intero.
 *
 * Restituisce il numero
 * 'int' rappresentato dalla stringa in argomento:
 * - saltando gli eventuali spazi bianchi iniziali,
 * - riconoscendo l'eventuale segno,
 * - fermandosi al raggiungimento del primo carattere non numerico.
 *
 * @param s Una stringa da convertire.
 * @return Il numero intero rappresentato nella stringa (il valore di ritorno non è
 * definito nel caso di un overflow).
 */

int stoi(char *s);
```

Esempio 2: Passo 2

- Noi scriviamo il file `sqr.c`
 - Contiene il `main` dell'applicazione
 - Usa le funzioni `square()` e `stoi()`
- Cosa sono `argc` e `argv`?
 - `argc`: numero di parametri sulla riga di comando
 - `argv`: array dei parametri
 - Ogni parametro è un stringa (`char *`)

`sqr.c`

```
#include <stdio.h>
#include <stdlib.h>
#include "square.h"
#include "stoi.h"

/**
 * @brief Accetta un intero sulla riga di comando e ne stampa il quadrato.
 *
 * @param argc Il numero di argomenti inseriti sulla riga di comando.
 * @param argv Un array di stringhe che memorizza gli argomenti inseriti.
 */
int main(int argc, char *argv[]) {
    if (argc != 2) { puts("`Usage: sqr number"); exit(EXIT_FAILURE); }
    int num = atoi(argv[1]);
    int res = square(num);
    printf("%d\n", res);
}
```


Esempio 2: Passo 3

- Compilazione di `sqr.c`
 - `gcc -c sqr.c`
- Linking di `sqr.o`, `stoi.o` e `square.o`
 - `gcc -o sqr sqr.o stoi.o square.o`
- Nota:
 - Non è necessario avere accesso al codice sorgente `stoi.c`. Bastano:
 - `stoi.h` che contiene la dichiarazione (come la funzione può essere usata, i.e., i parametri)
 - `stoi.o` che contiene l'implementazione in codice macchina

Mockup

- E se il collega non avesse ancora creato `stoi.o`?
 - Possiamo fare una demo dell'applicativo `sqr`?
 - Sì, usando un mockup
- **Mockup**:
 - Una funzione di facciata che simula la funzione vera
 - Ha stessa firma ... ma funzionalità limitata
 - Esempio:
 - Mockup di `stoi()` potrebbe essere una funzione che converte "-25", "+25" e "25" negli interi -25, 25 e 25, rispettivamente, mentre termina il programma per tutte le altre stringhe di input.



Test Driven Development (TDD)

Esempio TDD: Obiettivo

- Consideriamo il file `stoi.h`
 - Esso descrive il comportamento di `stoi()` in modo:
 - Informale
 - Linguaggio naturale misto a notazione matematica
 - Più possibile comprensibile e preciso
 - Evitando ambiguità
 - Più possibile completo
 - Include i casi limite
- e sviluppiamo per iterazioni successive la sua implementazione `stoi.c` che soddisfi la specifica in `stoi.h`

`stoi.h`

```
/*  
 * @brief Converte una stringa in intero.  
 *  
 * Restituisce il numero  
 * 'int' rappresentato dalla stringa in argomento:  
 * – saltando gli eventuali spazi bianchi iniziali,  
 * – riconoscendo l'eventuale segno,  
 * – fermandosi al raggiungimento del primo carattere non numerico.  
 *  
 * @param s Una stringa da convertire.  
 * @return Il numero intero rappresentato nella stringa (il valore di ritorno  
 * non è  
 * definito nel caso di un overflow).  
 */  
  
int stoi(char *s);
```

Test Suite

- Per testare `int stoi(char *s):`
 - Scriviamo un file `test_stoi.c`
- Contiene una collezione di test:
 - Test suite
 - Ogni test case è un **UNIT test**

UNIT Test

`test_stoi_v1.c`

```
// Test suite per stoi.c (Versione 1)
#include <stdio.h>
#include "stoi.h"

int main(void) {
    puts("BEGIN TEST stoi");
    /* IMPORTANTE: Ogni blocco codifica uno UNIT TEST
    isolato dagli altri */

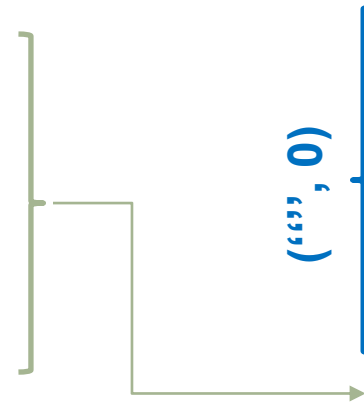
    {
        char *s = ""; /* stringa vuota */
        int n = stoi(s);
        if (0 != n) {
            printf("--TEST 1 FAILED--\n");
            printf("INPUT:%s\n", s);
            printf("EXPECTED:%d\n", 0);
            printf("RESULT:%d\n\n", n);
        }
    }

    /* ALTRI CASI a seguire */
}
```

Iterazione 1: test_stoi.c

- Scriviamo un file `test_stoi.c`
- Test per iterazioni successive:
 - Prima iterazione:
 - Scopo: verificare il comportamento di `stoi()` quando `s` contiene una sequenza di $n \geq 0$ caratteri '0'..'9', eventualmente seguita da altri caratteri non numerici. Nota: `s` può essere vuota.
 - Coppie (input, output) da testare:

• ("", 0)	• ("712abCZ", 712)
• ("0", 0)	• ("abCz12", 0)
• ("1", 1)	• ("012 43", 12)
• ("000420", 420)	• ("42t z", 42)
• ("327abZ", 327)	• ("z64", 0)
• ("315 ", 315)	



`test_stoi_v1.c`

```
// Test suite per stoi.c (Versione 1)
#include <stdio.h>
#include "stoi.h"

int main(void) {
    puts("BEGIN TEST stoi");
    /* IMPORTANTE: Ogni blocco codifica uno UNIT TEST
    isolato dagli altri */

    {
        char *s = ""; /* stringa vuota */
        int n = stoi(s);
        if !(0 == n) {
            printf("--TEST 1 FAILED--\n");
            printf("INPUT:%s\n", s);
            printf("EXPECTED:%d\n", 0);
            printf("RESULT:%d\n\n", n);
        }
    }

    /* ALTRI CASI a seguire */
}
```

Iterazione 1: `stoi.c`

- Scriviamo il file `stoi.c`
 - Versione 1 cerca di soddisfare `test_stoi_v1.c`

`test_stoi_v1.c`
è sufficiente?

- No:
 - Non copre tutte le specifiche
 - Continuiamo con la prossima iterazione

```
int stoi(char *s)
{
    int num = 0;
    int i = 0;
    // parsifica (e memorizza in num il valore di) un naturale decimale fino al
    // raggiungimento della fine della stringa s, o fino a quando incontra un
    // carattere non numerico.
    /* INVARIANTE DI CICLO: num contiene il naturale le cui cifre sono
    (nell'ordine) quelle in s[H..i-1]. */
    while (s[i] != '\0' && (s[i] >= '0' && s[i] <= '9'))
    {
        num = (num * 10) + (s[i] - '0');
        i++;
    }
    return num;
}
```

Iterazione 2: `test_stoi.c`

- Estendiamo `test_stoi_v1.c`
 - Seconda iterazione:
 - Scopo: verifica comportamento di `stoi()` quando `s` contiene un **eventuale carattere di segno** ('+' o '-') , seguito da una sequenza di $n \geq 0$ caratteri '0',.. '9', eventualmente seguita altri caratteri.
 - Coppie (input, output) aggiuntive da testare:

• ("+0", 0)	• ("-0", 0)
• ("+1", 1)	• ("-1", -1)
• ("+000420", 420)	• ("-000420", -420)
• ("+327abZ", 327)	• ("-327abZ", -327)
• ("+315 ", 315)	• ("-315 ", -315)
• ("+712abCZ", 712)	• ("-712abCZ", -712)
• ("+abCz12", 0)	• ("-abCz12", 0)
• ("+012 43", 12)	• ("-012 43", -12)
• ("+42t z", 42)	• ("-42t z", -42)
• ("+z64", 0)	• ("-z64", 0)

Iterazione 2: stoi.c

- Aggiorniamo il file `stoi.c`
 - Versione 2 cerca di soddisfare `test_stoi_v2.c`

Finito?

No: casi ancora da estendere

```
int stoi(char *s)
{
    int sign = 1; /* Il segno di default e' il + */
    int num = 0;
    int i = 0;
    /* ASSERZIONE: i contiene un valore H >= 0 tale che: s[0..H-1] contiene solo spazi e s[H] non e' uno spazio. */
    // aggiorna (eventualmente) il segno del numero
    if (s[i] == '+' || s[i] == '-')
    {
        if (s[i] == '-') { sign = -1; }
        i++;
    }
    /* ASSERZIONE: sign contiene il segno (+1 o -1); i contiene un valore H tale che s[0..H-1] contiene spazi
    eventualmente seguiti da un segno; se H > 0 allora s[H] != uno spazio. */
    // parsifica (e memorizza in num il valore di) un naturale decimale fino al raggiungimento della fine della
    stringa s,
    // o fino a quando incontra un carattere non numerico.
    /* INVARIANTE DI CICLO: num contiene il naturale le cui cifre sono (nell'ordine) quelle in s[H..i-1]. */
    while (s[i] != '\0' && (s[i] >= '0' && s[i] <= '9'))
    {
        num = (num * 10) + (s[i] - '0');
        i++;
    }
    return sign * num;
}
```

Iterazione 3: `test_stoi.c`

- Estendiamo `test_stoi_v2.c`
 - Terza iterazione:
 - Scopo: verifica comportamento di `stoi()` quando **s contiene spazi bianchi**, eventualmente seguiti dal carattere '+' o dal carattere '-', e dopo da $n \geq 0$ cifre decimali seguite da eventuali altri caratteri.
 - Coppie (input, output) aggiuntive da testare:

• (" +0", 0)	• (" -0", 0)
• (" +1", 1)	• (" -1", -1)
• (" +000420", 420)	• (" -000420", -420)
• (" +327abZ", 327)	• (" -327abZ", -327)
• (" +315 ", 315)	• (" -315 ", -315)
• (" +712abCZ", 712)	• (" -712abCZ", -712)
• (" +abCz12", 0)	• (" -abCz12", 0)
• (" +012 43", 12)	• (" -012 43", -12)
• (" +42t z", 42)	• (" -42t z", -42)
• (" +z64", 0)	• (" -z64", 0)

Iterazione 3: `stoi.c`

- Aggiorniamo il file `stoi.c`
 - Versione 3 cerca di soddisfare
`test_stoi_v3.c`

Finito?

Sì?

```
int stoi(char *s)
{
    int sign = 1; /* Il segno di default e' il + */
    int num = 0;
    int i = 0;
    // ignora gli eventuali spazi vuoti all'inizio di s
    while (s[i] == ' ' || s[i] == '\n' || s[i] == '\t') { i++; }
    /* ASSERTZIONE: i contiene un valore H >= 0 tale che: s[0..H-1] contiene solo spazi e s[H] non e' uno spazio. */
    // aggiorna (eventualmente) il segno del numero
    if (s[i] == '+' || s[i] == '-')
    {
        if (s[i] == '-') { sign = -1; }
        i++;
    }
    /* ASSERTZIONE: sign contiene il segno (+1 o -1); i contiene un valore H tale che s[0..H-1] contiene spazi
    eventualmente seguiti da un segno; se H > 0 allora s[H] != uno spazio. */
    // parsifica (e memorizza in num il valore di) un naturale decimale fino al raggiungimento della fine della
    stringa s,
    // o fino a quando incontra un carattere non numerico.
    /* INVARIANTE DI CICLO: num contiene il naturale le cui cifre sono (nell'ordine) quelle in s[H..i-1]. */
    while (s[i] != '\0' && (s[i] >= '0' && s[i] <= '9'))
    {
        num = (num * 10) + (s[i] - '0');
        i++;
    }
    return sign * num;
}
```

Regression Testing

- Riteniamo per il momento che il codice della versione 3 sia corretto
 - Solo fino a prova contraria
 - In caso di errori:
 - Li trasformiamo in test e li aggiungiamo alla test suite

Regression testing:

- Se dobbiamo estendere il comportamento del codice:
 1. Scriviamo i test che specificano il nuovo comportamento.
 2. Aggiorniamo il codice per soddisfare i nuovi test.
 3. Controlliamo che il codice continui a superare tutti i test precedenti.
- Il punto 3 serve ad evitare che il codice “regredisca”
 - Ovvero evitare che un aggiornamento introduca un errore per i test precedenti

Test Suite: Proprietà

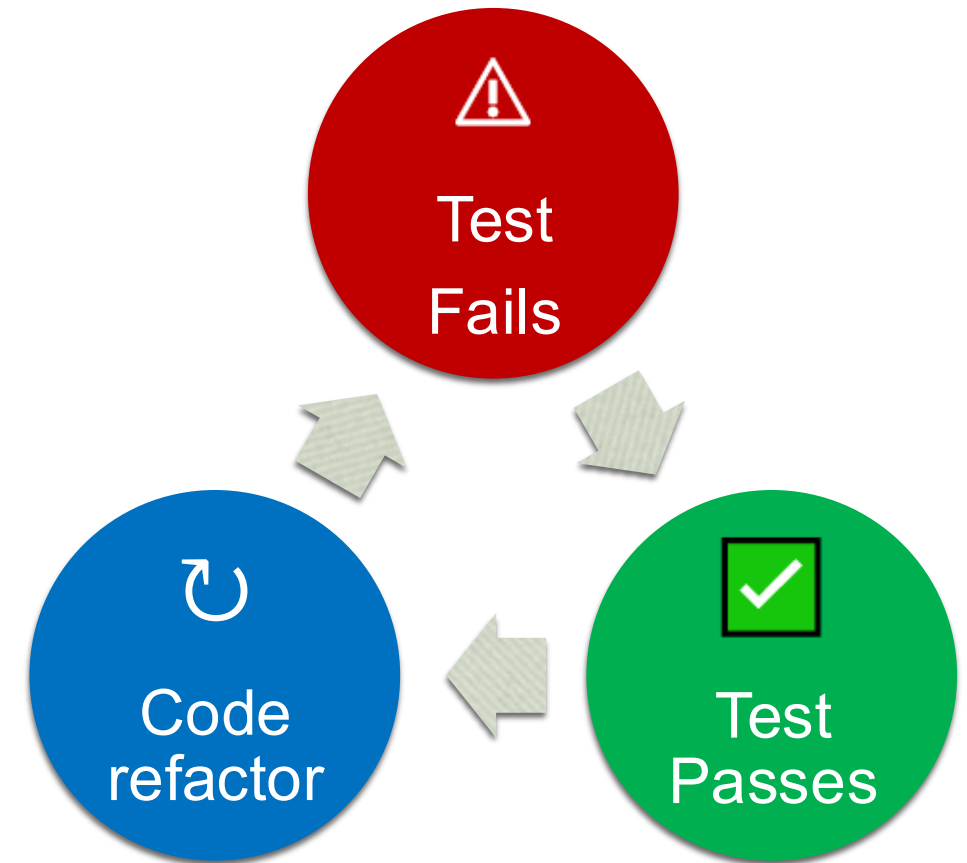
- La test suite di una funzione contiene una specifica:
 - **Formale**: scritta in un linguaggio di programmazione
 - **Precisa**: al più quanto lo è la specifica della funzione da testare
 - **Estendibile**: descrive delle coppie (input, output atteso)
 - **Incompleta**: salvo casi triviali, sarà sempre incompleta
 - **Significativa**: deve descrivere dei casi interessanti
 - **Verificabile automaticamente**: i test sono eseguibili
 - **In evoluzione**: guida lo sviluppo della funzione per iterazioni successive

Unit Test

- **UNIT test:**
 - Intende collaudare in isolamento il funzionamento di una singola unità
 - Per unità si intende il minimo componente di un programma dotato di funzionamento autonomo
 - Linguaggio C: unità = funzione
 - Scritti secondo i principi **FIRST²**:
 - **Fast**: veloci da eseguire (programmatore non esitano ad eseguirli)
 - **Isolated**: indipendente dal resto
 - **Repeatable**: ripetibili e deterministici
 - **Self-validating**: senza la necessità per il programmatore di verificare gli esiti
 - **Timely**: scritti tempestivamente, il prima possibile, anche prima del codice
 - **Thorough**: esaustivo per coprire tutti i casi di interesse

Test Driven Development I

- **Test Driven Development (TDD):**
 - In essenza è un processo iterativo in tre fasi
 1. Crea qualche test che fallisce
 - Anche solo di compilazione perché al codice di produzione manca una funzionalità
 2. Scrivi il minimo codice di produzione
 - Necessario per passare il test
 3. Refactor: rivedi e riorganizza il codice
 - Puoi renderlo più veloce/efficiente?
 - Puoi rimuovere qualche duplicazione?



Test Driven Development II

- Le regole d'oro:
 - Non scrivere codice finché un test non lo richiede
 - Fai riferimento a nuove funzioni, variabili, ecc. prima nel test in modo che il codice non compili, e poi correggi dichiarandole
 - Cerca di avere una sola cosa rotta alla volta, se possibile
- Documentazione:
 - Nell'approccio TDD, gli unit test fungono da documentazione del codice.

Test Driven Development III

- Costruire la cosa giusta.
 - Usiamo i test come specifiche per ciò che vogliamo fare con il software.
- Assicurarsi che il software funzioni sempre.
 - Automatizziamo i test per eseguirli rapidamente. In questo modo, dopo ogni piccola modifica, possiamo ritestare il software per assicurarci che funzioni ancora.
- Mantenere il ritmo dello sviluppo.
 - Man mano che il codice cresce, diventa sempre più difficile modificarlo senza romperlo. Mantenere il codice sempre funzionante significa che possiamo consegnare il software prima. Aggiungere nuove funzionalità a un nuovo software è facile e la nostra produttività iniziale è elevata.
- Affidabilità vs. Produttività
 - Un bug scoperto dagli utenti in produzione può costare 100 volte di più da correggere rispetto a se fosse stato individuato non appena il programmatore ha introdotto l'errore.